PROJECT REPORT

(Based on Summer Internship Training)

AI RESUME ANALYZER & ATS PLATFORM

An AI-Powered Applicant Tracking System Built on the MERN Stack

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

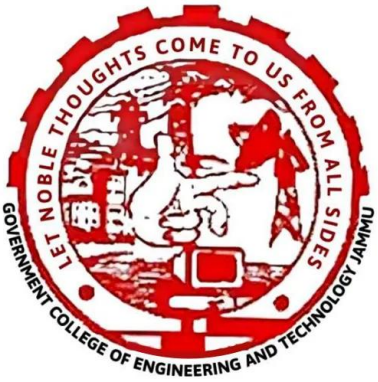
By

Ankur Sharma (231103008)

Under the Supervision of

Mr. Chetan Salmotra

(Senior Software Development Engineer, Maincrafts Technology)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

GOVERNMENT COLLEGE OF ENGINEERING AND TECHNOLOGY, JAMMU

Academic Year 2023 – 2027

CERTIFICATE OF INTERNSHIP



The internship undertaken at **Maincrafts Technology**, as evidenced by the certificate above, formed the practical foundation for the major project documented in this report. The candidate completed a **six-week online summer internship** (01 July 2026 to 01 August 2026) in **Artificial Intelligence and Full-Stack Web Development**, during which the AI Resume Analyzer & ATS Platform was conceived, designed, and implemented under the mentorship of Mr. Chetan Salmotra, Senior Software Development Engineer.

DECLARATION BY CANDIDATE

I hereby declare that the work presented in this report entitled **"AI RESUME ANALYZER & ATS PLATFORM"** in fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, submitted in the Department of Computer Science & Engineering, **Government College of Engineering and Technology, Jammu**, is an authentic record of my own work carried out at **Maincrafts Technology** over the period from July 2026 to August 2026, under the supervision of **Mr. Chetan Salmotra**.

I further declare that this report has not been submitted, in part or full, to any other university or institute for the award of any other degree or diploma, and that all sources of information used in this report have been duly acknowledged.

(Student's Signature)

Ankur Sharma

231103008

This is to certify that the above statement made by the candidate is true to the best of my knowledge.

Mr. Chetan Salmotra

(Senior Software Development Engineer)

Maincrafts Technology

ACKNOWLEDGEMENT

I take this opportunity to express my sincere thanks and deep gratitude to all those who extended their wholehearted cooperation and helped me in completing this internship and project successfully.

First and foremost, I would like to thank **Mr. Chetan Salmotra**, my mentor at Maincrafts Technology, who guided me, mentored me, and challenged me throughout the development of this project. Last but not the least, I would like to thank the founders of Maincrafts Technology for considering me a part of their organization and providing such a great platform to learn and enhance my skills.

A very special thanks goes to all the faculty members of the Department of Computer Science & Engineering, Government College of Engineering and Technology, Jammu, under whose guidance I have been able to excel in my academic career and successfully undertake this project.

Finally, I am deeply grateful to my family and friends for their unwavering support and encouragement throughout this journey.

Ankur Sharma

(CSE, 231103008)

Government College of Engineering and Technology, Jammu

ABSTRACT

Recruitment in the modern technology industry faces a persistent bottleneck: the manual screening of resumes against job requirements. As the volume of applications received for any given opening continues to rise, recruiters are forced to spend an increasing share of their working time reading unstructured resume documents and comparing candidate qualifications against job descriptions. This process is time-consuming, inconsistent, and susceptible to human fatigue and bias.

This project, "**AI Resume Analyzer & ATS Platform**," presents a full-stack, AI-driven Applicant Tracking System (ATS) that automates and objectifies the resume-screening process. Built on the MERN stack (MongoDB, Express.js, React, Node.js) and augmented with Google's Gemini Generative AI model, the platform allows a candidate to upload a resume in PDF format, which is parsed using the pdf-parse library. The extracted text, together with a target job description, is passed through a carefully engineered prompt to the Gemini AI model, which returns a structured JSON assessment containing extracted skills, work experience, education, strengths, weaknesses, an ATS compatibility score, a job-match percentage, matched and missing skills, and actionable improvement suggestions.

On the recruiter-facing side, the platform offers a real-time analytics dashboard built with Recharts, visualizing application trends, skill distribution, and hiring statistics. A centralized candidate database stores every analysis performed, enabling recruiters to rank and shortlist applicants by objective, AI-derived compatibility scores.

The frontend is implemented using React 19 with Vite 7, styled with TailwindCSS 4 and animated with Framer Motion, while SWR handles client-side data fetching and caching. The backend exposes a secure, modular REST API built with Express.js 5, backed by MongoDB via Mongoose. The application has been fully deployed, with the frontend hosted on Vercel (https://client-blue-xi.vercel.app) and the backend hosted on Render (https://ai-resume-analyzer-22f8.onrender.com), demonstrating a complete, production-ready, cloud-deployed software system.

This report documents the complete engineering lifecycle of the system — requirement analysis, system design, database modelling, implementation, AI prompt engineering, testing, and results — and demonstrates the practical application of modern web technologies combined with generative AI to solve a genuine, high-impact problem in recruitment technology.

Keywords: Applicant Tracking System, Artificial Intelligence, Google Gemini AI, Resume Parsing, Natural Language Processing, MERN Stack, Recruitment Technology, ATS Score.

TABLE OF CONTENTS

Certificate of Internship	**2**
Declaration by Candidate	**3**
Acknowledgement	**4**
Abstract	**5**
List of Figures	**9**
List of Tables	**10**
Abbreviations	**11**
CHAPTER 1: INTRODUCTION	**12**
1.1 Background	12
1.2 Motivation	12
1.3 Problem Statement	13
1.4 Existing System and Its Limitations	13
1.5 Proposed System	13
1.6 Objectives	13
1.7 Scope	14
1.8 Applications and Advantages	14
1.9 Organization of the Report	14
CHAPTER 2: LITERATURE REVIEW	**15**
2.1 Applicant Tracking Systems	15
2.2 AI-Driven Recruitment and Resume Parsing	15
2.3 Google Gemini AI	15
2.4 Comparative Analysis	15
2.5 Research Gap	16
CHAPTER 3: SYSTEM ANALYSIS	**17**
3.1 Functional Requirements	17
3.2 Non-Functional Requirements	17
3.3 Software and Hardware Requirements	17
3.4 Feasibility Study	18
3.5 Risk Analysis	18
CHAPTER 4: SYSTEM DESIGN	**19**
4.1 High-Level System Architecture	19
4.2 System Flow Chart	19
4.3 Entity Relationship Diagram (Conceptual)	20
4.4 Sequence Diagram – Resume Analysis Workflow	20
CHAPTER 5: DATABASE DESIGN	**21**
5.1 Introduction	21
5.2 Collection Schemas	21

5.3 Sample Analysis Document	22
5.4 Indexing Strategy	22
CHAPTER 6: IMPLEMENTATION	**24**
6.1 Project Folder Structure	24
6.2 Backend Implementation	24
6.3 REST API Endpoint Summary	25
6.4 Frontend Implementation	26
6.5 Performance Optimization	26
CHAPTER 7: AI MODULE	**27**
7.1 AI Resume Analysis Pipeline	27
7.2 Prompt Engineering	27
7.3 Calling the Gemini API	28
7.4 Job Matching Algorithm	28
7.5 AI Limitations	28
CHAPTER 8: TESTING	**29**
8.1 Unit and Integration Testing	29
8.2 Manual and Exploratory Testing	29
8.3 Performance Testing	29
8.4 Security Testing	29
CHAPTER 9: RESULTS	**31**
9.1 Dashboard	31
9.2 Resume Analyzer – Upload	31
9.3 Resume Analyzer – Processing State	32
9.4 Job Match Analysis	32
9.5 Candidate Database	33
9.6 Job Postings	33
9.7 Analytics & Insights	34
9.8 Job Finder	35
9.9 Account Settings	35
9.10 Discussion of Results	36
CHAPTER 10: ADVANTAGES, LIMITATIONS, FUTURE SCOPE,	**37**
SOCIAL IMPACT AND CONCLUSION	37
10.1 Advantages	37
10.2 Limitations	37
10.3 Future Scope	37
10.4 Social Impact	37
10.5 Conclusion	38
REFERENCES	**39**

LIST OF FIGURES

Figure No.	Caption
4.1	High-Level System Architecture
4.2	System Flow Chart
4.3	Entity Relationship Diagram (Conceptual)
4.4	Sequence Diagram – Resume Analysis Workflow
7.1	AI Resume Analysis Pipeline
9.1	Dashboard – Recruitment Overview
9.2	Resume Analyzer – Upload Screen
9.3	Resume Analyzer – Processing State
9.4	Job Match Analysis – ATS Result
9.5	Candidate Database
9.6	Job Postings
9.7	Analytics & Insights (Trends)
9.8	Analytics & Insights (Top Skills)
9.9	Job Finder
9.10	Account Settings

LIST OF TABLES

Table No.	Caption
2.1	Comparative Analysis of Existing ATS Solutions
3.1	Functional Requirements
3.2	Non-Functional Requirements
3.3	Software and Hardware Requirements
3.4	Risk Analysis Matrix
5.1	Users Collection Schema
5.2	Candidates Collection Schema
5.3	Jobs Collection Schema
5.4	Analysis Collection Schema
6.1	REST API Endpoint Summary
8.1	Unit and Integration Test Cases
8.2	Security Test Cases

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ATS	Applicant Tracking System
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ER	Entity Relationship
HR	Human Resources
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MERN	MongoDB, Express.js, React, Node.js
NLP	Natural Language Processing
ODM	Object Data Modelling
PDF	Portable Document Format
REST	Representational State Transfer
SDK	Software Development Kit
SPA	Single Page Application
SWR	Stale-While-Revalidate
UI/UX	User Interface / User Experience

CHAPTER 1: INTRODUCTION

1.1 Background

Recruitment is one of the most critical functions within any organization, directly influencing the quality of talent an organization can attract and retain. Traditionally, this process has relied heavily on manual effort: recruiters read through resumes submitted in response to job postings, compare the candidate's listed skills and experience against the requirements of the open position, and shortlist candidates for further rounds of interviews.

With the growth of online job portals and remote work opportunities, the number of applications received for a single job posting has grown substantially. Manually reviewing this volume of resumes is impractical and prone to inconsistency, since different reviewers may apply different, often implicit, criteria when evaluating the same resume.

To address this challenge, organizations adopted Applicant Tracking Systems (ATS) — software platforms designed to manage the recruitment and hiring process. Early ATS platforms relied primarily on rule-based keyword matching: a resume was scanned for the presence or absence of specific keywords drawn from the job description, and a match score was computed based on simple keyword overlap. While this improved upon fully manual screening, keyword-matching algorithms lack semantic understanding — they cannot recognize that differently worded skill descriptions refer to the same underlying competency.

The emergence of Large Language Models (LLMs), such as Google's Gemini family of generative AI models, presents an opportunity to overcome this limitation. This project, AI Resume Analyzer & ATS Platform, integrates Google Gemini AI directly into a full-stack MERN web application to deliver an ATS platform that performs semantic, context-aware resume analysis rather than superficial keyword matching.

1.2 Motivation

- The screening bottleneck is a genuine, widely reported problem — recruiters spend a disproportionate share of their time on initial resume screening rather than on higher-value activities such as interviewing and candidate engagement.
- Existing ATS tools are largely rule-based and rigid, frequently criticized for producing inconsistent or unfair results, particularly for candidates who describe similar skills using different terminology.
- Generative AI models have matured to a point where they can be reliably integrated into production applications through well-documented, cost-effective APIs such as the Google Generative AI SDK, making it feasible to build a genuinely intelligent NLP-driven application without training a custom model from scratch.

These observations motivated the author to design and build a project that is both academically substantial — touching on full-stack engineering, database design, REST API architecture, and applied

natural language processing — and practically useful, solving a real problem faced by recruiters and job seekers alike.

1.3 Problem Statement

"To design and develop a full-stack, AI-powered Applicant Tracking System that automates resume parsing and evaluation, objectively measures the semantic compatibility between a candidate's resume and a given job description using a generative AI model, and provides both candidates and recruiters with actionable, data-driven insights through an intuitive web interface and real-time analytics dashboard."

1.4 Existing System and Its Limitations

Prior to AI-driven ATS platforms, the recruitment screening landscape was dominated by fully manual screening (in smaller organizations) and traditional rule-based ATS platforms (in larger organizations) that parse resumes using regular-expression or dictionary-based keyword extraction, compute match scores by counting keyword overlap, and rank candidates purely by keyword count. The principal limitations of these systems are:

- Lack of semantic understanding — synonymous or paraphrased skills are not recognized.
- High false-negative rate — qualified candidates are filtered out due to wording or formatting mismatches.
- No actionable feedback for candidates — traditional ATS platforms are recruiter-facing only.
- Static, non-contextual scoring that cannot infer experience depth or project relevance.
- Poor analytics, particularly regarding skill-distribution trends across the applicant pool.
- High cost and vendor lock-in for commercial ATS platforms.

1.5 Proposed System

The proposed AI Resume Analyzer & ATS Platform directly addresses each of these limitations by combining a modern full-stack web architecture with a generative AI reasoning layer. Rather than counting keyword occurrences, the system sends extracted resume text and the job description together to the Google Gemini AI model with a carefully engineered prompt, requesting a structured JSON response containing extracted skills, experience, education, ATS compatibility score, match percentage, matched/missing skills, and improvement suggestions. The platform is useful to both recruiters (via the dashboard, candidate database, and job-postings module) and candidates (via the ATS score, missing-skill feedback, and Smart Job Finder) and is built entirely on open-source frameworks combined with a pay-as-you-go AI API, making it considerably more affordable and extensible than commercial ATS software.

1.6 Objectives

1. To design and implement a secure user authentication and account settings system.

2. To implement a reliable resume-upload pipeline supporting PDF validation and drag-and-drop upload.
3. To implement server-side resume text extraction using the pdf-parse Node.js library.
4. To integrate the Google Gemini Generative AI SDK to perform structured resume analysis — skill extraction, experience/education parsing, strengths/weaknesses identification, and ATS compatibility scoring.
5. To design a job-description matching module that computes a quantitative match score and lists matched/missing skills.
6. To design a MongoDB database capable of storing users, candidates, job postings, and AI-generated analysis records.
7. To build a recruiter-facing analytics dashboard visualizing application trends, skill distribution, and hiring statistics.
8. To implement a Smart Job Finder that recommends relevant job postings to a candidate based on extracted skills.
9. To implement a candidate database and job-posting module supporting CRUD operations.
10. To deploy the completed application to production using Vercel and Render.

1.7 Scope

The scope of this project encompasses the complete design, implementation, testing, and deployment of a web-based ATS platform: a responsive, dark-themed React frontend; a RESTful Node.js/Express.js backend; integration with Google Gemini AI for natural language analysis of resumes; a MongoDB data layer; and deployment to a live, publicly accessible production environment. The scope explicitly excludes video/audio-based candidate interviews, integration with third-party job boards for automatic job scraping, payroll or onboarding functionality, and support for resume formats other than PDF in the current version.

1.8 Applications and Advantages

The platform has direct applicability to corporate recruitment teams automating high-volume screening, startups and small businesses that cannot afford commercial ATS licenses, campus placement cells, and individual job seekers who wish to independently evaluate and improve their resumes. Key advantages include a significant reduction in manual screening time, objective and consistent AI-generated compatibility scores, actionable candidate feedback, a real-time visual analytics dashboard, and a modern, scalable, cost-effective technology stack.

1.9 Organization of the Report

The remainder of this report is organized as follows:

Chapter 2 surveys existing research and commercial systems relevant to AI-driven recruitment. Chapter 3 details the requirement analysis, feasibility study, and risk analysis.

Chapter 4 presents the system architecture and design diagrams.

Chapter 5 describes database design.

Chapter 6 explains the implementation.

Chapter 7 provides an in-depth look at the AI module.

Chapter 8 documents the testing performed.

Chapter 9 presents the results with screenshots from the deployed application.

Chapter 10 discusses advantages, limitations, future scope, social impact, and the overall conclusion, followed by the References.

CHAPTER 2: LITERATURE REVIEW

2.1 Applicant Tracking Systems

Applicant Tracking Systems emerged in the late 1990s and early 2000s as organizations sought to digitize the collection and organization of job applications. Commercial platforms such as Taleo, iCIMS, Greenhouse, and Workday Recruiting expanded this foundation by adding workflow automation. Despite these improvements, the underlying resume-to-job matching logic in most commercial ATS platforms has historically remained keyword-centric, which has led to the well-known practice of "resume keyword stuffing" as a job-seeker workaround.

2.2 AI-Driven Recruitment and Resume Parsing

Research interest in applying machine learning to recruitment has grown substantially, generally exploring classification-based candidate ranking (supervised models trained on historical hiring data) and semantic similarity scoring (comparing vector representations of resumes and job descriptions using techniques such as TF-IDF, Word2Vec, or transformer-based embeddings). Resume parsing itself has evolved from rule-based/regular-expression parsing, through statistical Named Entity Recognition (NER), to the most recent approach — used in this project — of prompting a large language model to read raw extracted text and return a structured JSON object directly, combining entity recognition, summarization, and contextual reasoning into a single inference step.

2.3 Google Gemini AI

Google Gemini is a family of multimodal, instruction-tuned large language models developed by Google DeepMind, accessible to developers through the Google Generative AI SDK. Gemini models support long-context input (allowing a full resume and job description in a single prompt), exhibit strong instruction-following behavior that enables reliable, structured JSON output, and are exposed through a well-documented, cost-effective API — properties that make the model well suited to the structured-extraction use case central to this project.

2.4 Comparative Analysis

Table 2.1 compares representative existing systems and research directions against the system proposed in this report.

Table 2.1: Comparative Analysis of Existing ATS Solutions and Research Work

System / Approach	Matching Approach	Semantic Understanding	Candidate Feedback	Analytics Dashboard
Traditional keyword-based ATS	Boolean keyword search	No	No	Limited (funnel only)
Commercial cloud ATS platforms	Keyword + manual tagging	Partial	No	Yes (funnel/pipeline)
TF-IDF resume-JD similarity research	Vector-space cosine similarity	Partial (lexical)	No	No

System / Approach	Matching Approach	Semantic Understanding	Candidate Feedback	Analytics Dashboard
NER-based resume parsers	Named entity extraction	Partial	No	No
Consumer resume-scoring extensions	Keyword density scoring	No	Yes (score only)	No
Proposed System (this project)	Structured LLM (Gemini) analysis	Yes	Yes (structured, actionable)	Yes (real-time, Recharts)

2.5 Research Gap

The comparative analysis highlights a clear research gap: very few systems combine genuine semantic LLM-based matching, structured actionable candidate feedback, a recruiter-facing real-time analytics dashboard, and a persistent, query able candidate database within a single, deployed, end-to-end platform. This project addresses that gap by unifying these capabilities within a single MERN-based, Gemini-AI-powered platform.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Functional Requirements

Table 3.1: Functional Requirements

ID	Requirement	Description
FR-01	User Authentication	Allow users to sign up, log in, and manage account settings.
FR-02	Resume Upload	Allow users to upload a resume in PDF format via drag-and-drop or file-picker.
FR-03	Resume Text Extraction	Extract raw text content from the uploaded PDF using pdf-parse.
FR-04	AI Resume Analysis	Submit extracted resume text (and job description) to Gemini AI and receive a structured analysis.
FR-05	ATS Compatibility Score	Compute and display a percentage-based ATS compatibility score.
FR-06	Job Description Matching	Compute a match score, matched skills list, and missing skills list against a job description.
FR-07	Candidate Database	Persist for each analysis in a searchable, sortable candidate database.
FR-08	Recruiter Dashboard	Display charts of application trends, skills distribution, and candidate statistics.
FR-09	Job Postings Management	Allow creation and management of job postings.
FR-10	Smart Job Finder	Recommend relevant open job postings to a candidate based on extracted skills.

3.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Category	Requirement
Performance	Resume analysis should be completed within an acceptable time window, depending on Gemini AI API latency.
Usability	The interface shall be intuitive, responsive, and require no user training.
Reliability	The system shall gracefully handle malformed PDFs and AI API failures without crashing.
Security	Protected endpoints shall require valid authentication; file uploads shall be validated.
Scalability	The stateless REST API and document-based database shall support horizontal scaling.
Portability	The frontend shall function correctly across major modern browsers and screen sizes.
Maintainability	The codebase shall follow a modular structure (routes, controllers, services).

3.3 Software and Hardware Requirements

Table 3.3: Software and Hardware Requirements

Component	Specification
Frontend	React 19, Vite 7, Tailwind CSS 4, Framer Motion, Recharts, SWR, React Router DOM v7
Backend	Node.js, Express.js 5, MongoDB, Mongoose, Multer, pdf-parse

Component	Specification
AI Service	Google Gemini AI via @google/generative-ai SDK
Version Control	Git and GitHub
Deployment	Vercel (frontend), Render (backend)
Development Machine	Dual-core 2 GHz+ processor, 8 GB RAM minimum, 10 GB free disk space
Client Browser	Latest versions of Chrome, Firefox, Edge, or Safari

3.4 Feasibility Study

Economic Feasibility: The project uses open-source frameworks with no licensing cost; the only recurring cost is usage-based billing for the Gemini AI API, which offers a free tier sufficient for development and low-volume production use. Technical Feasibility: All required technologies are mature, well-documented, and widely supported. Operational Feasibility: The system requires no specialized training for end users. Legal Feasibility: The system relies exclusively on data voluntarily submitted by users and processes it through a reputable, publicly documented third-party AI API under its published terms of service.

3.5 Risk Analysis

Table 3.4: Risk Analysis Matrix

Risk	Likelihood	Impact	Mitigation
Gemini API downtime / rate-limiting	Medium	High	Retries, graceful error messages, fallback UI states
Malformed / corrupted PDF uploads	Medium	Medium	Server-side file-type and content validation
Inconsistent AI JSON output	Medium	Medium	Strict prompt engineering and defensive response parsing
Unauthorized access to protected routes	Low	High	Token-based authentication middleware
Scalability bottleneck under load	Low	Medium	Stateless REST API enabling horizontal scaling

CHAPTER 4: SYSTEM DESIGN

4.1 High-Level System Architecture

The system follows a three-tier client-server architecture: a presentation tier (React SPA), an application/logic tier (Express.js REST API, including the AI service layer), and a data tier (MongoDB).

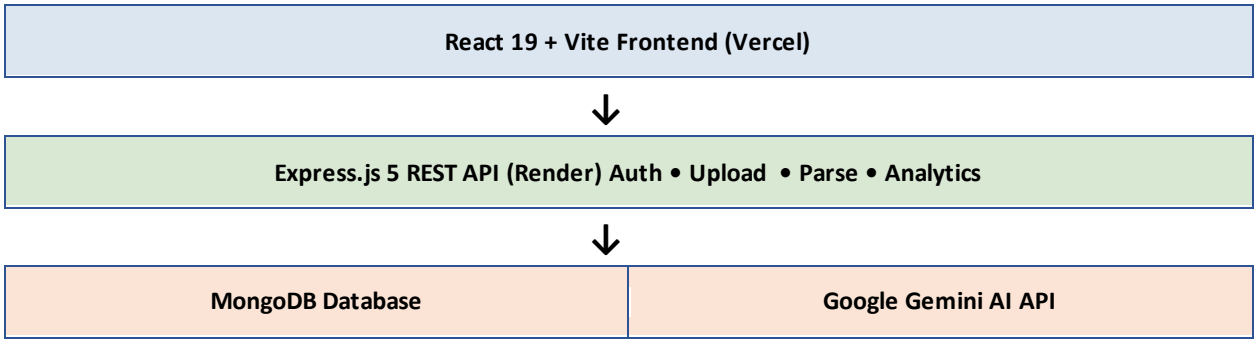


Figure 4.1: High-Level System Architecture

The React frontend, deployed on Vercel, communicates with the Express.js backend, deployed on Render, exclusively via HTTPS REST API calls. The backend authenticates requests, handles file uploads via Multer, extracts resume text via pdf-parse, and delegates semantic analysis to the Gemini AI service. All persistent data is stored in MongoDB via Mongoose.

4.2 System Flow Chart

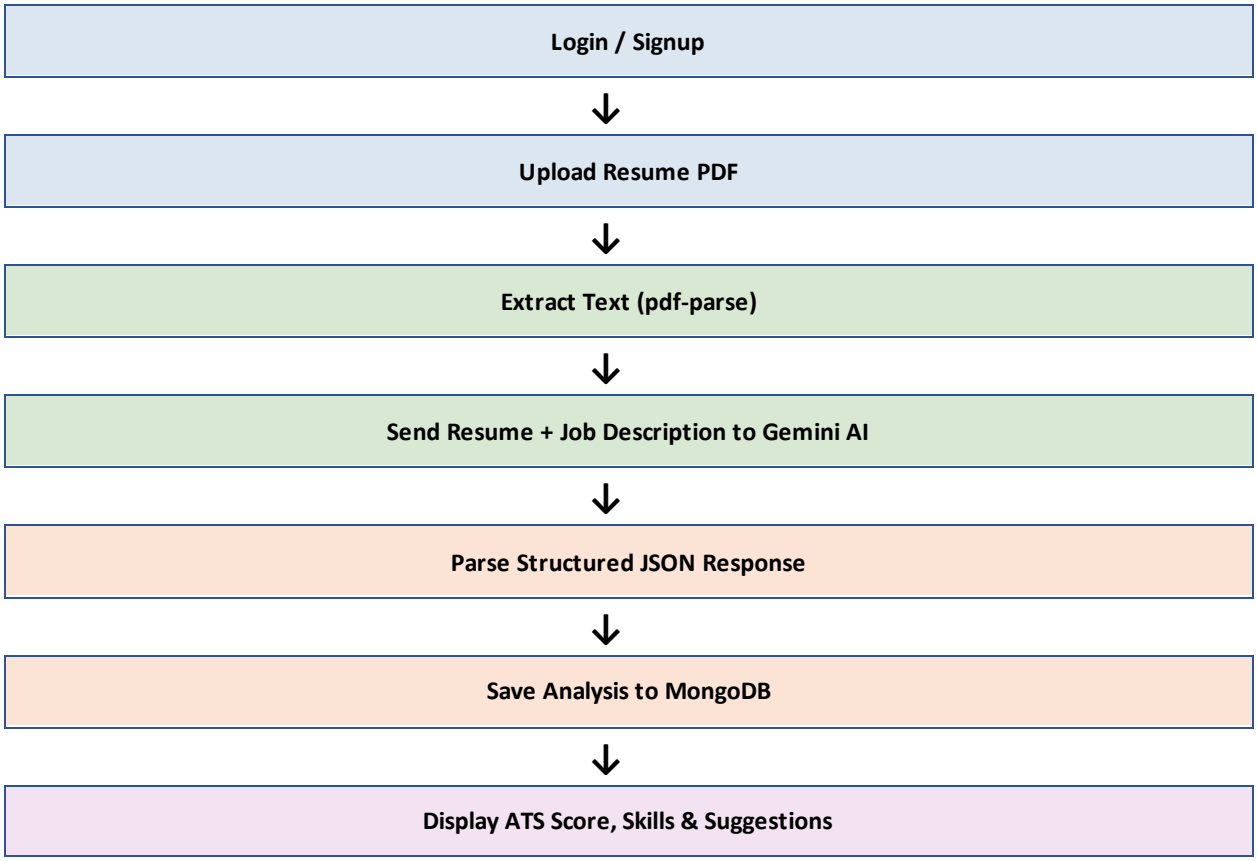


Figure 4.2: System Flow Chart

This flowchart traces the primary user journey from authentication through resume upload, AI analysis, and result persistence, to the display of the final assessment on the dashboard.

4.3 Entity Relationship Diagram (Conceptual)

The database is organized around four related entities: Users, Candidates, Jobs, and Analysis records.

Entity	Related To	Relationship
User	Candidate	One user can upload many candidate/resume records (1: N)
Candidate	Analysis	One candidate record has one or more analysis records over time (1: N)
Job	Analysis	One job posting can be referenced by many analysis records (1: N)

Figure 4.3: Entity Relationship Diagram (Conceptual)

4.4 Sequence Diagram – Resume Analysis Workflow

The sequence of interactions during a resume analysis request proceeds as follows:

1. Candidate selects a resume PDF and (optionally) pastes a job description in the React frontend.
2. Frontend sends a multipart/form-data POST request to /api/resume/analyze on the Express backend.
3. Multer middleware validates and buffers the uploaded file.
4. The Resume Parser Service extracts raw text from the PDF using pdf-parse.
5. The Gemini AI Service constructs a structured prompt and calls generateContent() on the Gemini API.
6. The Gemini API returns a structured JSON response containing skills, scores, and suggestions.
7. The backend validates and parses the JSON response.
8. The backend persists in a Candidate record and an Analysis record in MongoDB.
9. The backend returns a 200 OK response with the complete analysis to the frontend.
10. The frontend renders the ATS score, skill breakdown, and recommendations to the candidate.

Figure 4.4: Sequence Diagram – Resume Analysis Workflow (Textual Representation)

CHAPTER 5: DATABASE DESIGN

5.1 Introduction

The AI Resume Analyzer & ATS Platform uses MongoDB, a document-oriented NoSQL database, accessed through the Mongoose Object Data Modelling (ODM) library. MongoDB was selected because the AI-generated analysis output is itself a nested, semi-structured JSON object, which maps naturally onto MongoDB's flexible document model without requiring extensive normalization, and because the project already uses JavaScript/JSON end-to-end.

5.2 Collection Schemas

5.2.1 Users Collection

Table 5.1: Users Collection Schema

Field	Type	Description
_id	ObjectId	Unique document identifier (Primary Key)
name	String	Full name of the user
email	String	Login email address (unique, indexed)
password	String	Hashed password
role	String	candidate | recruiter | admin
createdAt	Date	Account creation timestamp

5.2.2 Candidates Collection

Table 5.2: Candidates Collection Schema

Field	Type	Description
_id	ObjectId	Unique document identifier (Primary Key)
userId	ObjectId	Reference to the Users collection
resumeFileName	String	Original uploaded file name
skills	[String]	Flattened list of extracted skills (denormalized for analytics)
atsScore	Number	Latest ATS compatibility score (0–100)
uploadedAt	Date	Upload timestamp

5.2.3 Jobs Collection

Table 5.3: Jobs Collection Schema

Field	Type	Description
_id	ObjectId	Unique document identifier (Primary Key)
title	String	Job title
company	String	Hiring company / department name
description	String	Full job description text

Field	Type	Description
requiredSkills	[String]	Skills required for the role (indexed)
applyUrl	String	External application URL (optional)
postedAt	Date	Posting timestamp

5.2.4 Analysis Collection

Table 5.4: Analysis Collection Schema

Field	Type	Description
_id	ObjectId	Unique document identifier (Primary Key)
candidateId	ObjectId	Reference to the Candidates collection
jobId	ObjectId	Reference to the Jobs collection (optional)
skills	[String]	Skills extracted by Gemini AI
strengths / weaknesses	[String]	AI-identified resume strengths and weaknesses
atsScore	Number	ATS compatibility percentage (0–100)
matchScore	Number	Job-description match percentage
matchedSkills / missingSkills	[String]	Skills present/absent relative to the job description
suggestions	[String]	AI-generated improvement suggestions
analyzedAt	Date	Timestamp of analysis

5.3 Sample Analysis Document

```
{
  "candidateId": "665f1e4a8a1b2c3d4e5f6790",
  "skills": ["React", "Node.js", "MongoDB", "JavaScript"],
  "atsScore": 76,
  "matchScore": 76,
  "matchedSkills": ["React", "Node.js"],
  "missingSkills": ["TypeScript", "Docker", "System Design", "Kubernetes"],
  "suggestions": [
    "Add experience with Docker containers.",
    "Include cloud infrastructure tools.",
    "Mention system design architectures explicitly."
  ],
  "analyzedAt": "2026-07-24T12:13:00.000Z"
}
```

5.4 Indexing Strategy

- Users: unique index on email for fast login lookup and duplicate registration prevention.
- Candidates: index on userId for fast retrieval of a user's uploaded resumes; index on atsScore (descending) for dashboard ranking.
- Jobs: Multikey index on requiredSkills for fast job recommendation queries.
- Analysis: index on candidateId and jobId for fast history and per-job retrieval.

CHAPTER 6: IMPLEMENTATION

6.1 Project Folder Structure

The repository is organized as a monorepo with two top-level application directories: client (React frontend) and server (Node.js/Express backend).

```
ai-resume-analyzer/
├── client/                 React 19 + Vite frontend
│   └── src/
│       ├── components/     Resume upload, dashboard charts, job cards
│       ├── pages/          Dashboard, Analyzer, Candidates, Jobs, Settings
│       ├── hooks/          SWR data-fetching hooks
│       ├── context/        AuthContext for session state
│       └── services/       API wrapper functions
└── server/                 Node.js + Express backend
    └── src/
        ├── controllers/    Request handlers (auth, resume, job, analytics)
        ├── routes/         Express route definitions
        ├── models/         Mongoose schemas (User, Candidate, Job, Analysis)
        ├── middleware/     Auth guard, Multer upload config
        └── services/       geminiService.js, resumeParserService.js
```

6.2 Backend Implementation

6.2.1 File Upload Middleware (Multer)

```
const multer = require('multer');
const storage = multer.memoryStorage();

function pdfFileFilter(req, file, cb) {
  if (file.mimetype === 'application/pdf') cb(null, true);
  else cb(new Error('Only PDF files are allowed'), false);
}

const upload = multer({
  storage,
  fileFilter: pdfFileFilter,
  limits: { fileSize: 5 * 1024 * 1024 } // 5 MB limit
});
module.exports = upload;
```

Multer is configured with in-memory storage, a strict MIME-type filter that only accepts application/pdf, and a 5 MB file-size cap. The resulting file buffer is passed directly to the PDF parsing service without ever touching disk.

6.2.2 Resume Parser Service

```
const pdfParse = require('pdf-parse');

async function extractText(fileBuffer) {
  const data = await pdfParse(fileBuffer);
  return cleanText(data.text);
}
```

```
function cleanText(rawText) {
  return rawText.replace(/\s+/g, ' ').replace(/\u0000/g, '').trim();
}
module.exports = { extractText, cleanText };
```

6.2.3 Resume Controller

```
async function analyzeResume(req, res, next) {
  try {
    const { jobDescription } = req.body;
    if (!req.file) return res.status(400).json({ message: 'Resume file is required' });

    const resumeText = await extractText(req.file.buffer);
    const aiResult = await generateAnalysis(resumeText, jobDescription);

    const candidate = await Candidate.create({
      userId: req.user.id,
      resumeFileName: req.file.originalname,
      skills: aiResult.skills,
      atsScore: aiResult.atsScore
    });

    const analysis = await Analysis.create({ candidateId:
candidate._id, ...aiResult });
    return res.status(200).json({ candidate, analysis });
  } catch (err) { next(err); }
}
```

This controller orchestrates the full resume-analysis pipeline: it validates the uploaded file, extracts its text, delegates to the Gemini AI service for semantic analysis, persists both a Candidate record and a detailed Analysis record, and returns the combined result to the frontend.

6.3 REST API Endpoint Summary

Table 6.1: REST API Endpoint Summary

Method	Endpoint	Description
POST	/api/auth/signup	Register a new user account.
POST	/api/auth/login	Authenticates a user and returns a session token.
POST	/api/resume/analyze	Uploads a PDF resume (and optional job description), returns AI analysis.
GET	/api/candidates	Retrieves the paginated candidate database.
GET	/api/job-finder	Returns recommended jobs based on the candidate's extracted skills.
GET / POST	/api/jobs	Lists / create job postings.
GET	/api/analytics	Aggregates and serves dashboard metrics.

6.4 Frontend Implementation

The frontend is organized around React Router DOM v7 for client-side routing, React Context for global authentication state, and SWR for server-state fetching and caching. The Resume Upload component implements drag-and-drop with client-side type/size pre-validation, mirroring the server-side Multer validation as a fast-fail UX improvement.

```
function SkillsDistributionChart({ data }) {
  return (
    <ResponsiveContainer width="100%" height={300}>
      <BarChart data={data}>
        <XAxis dataKey="skill" /><YAxis /><Tooltip />
        <Bar dataKey="count" radius={[6, 6, 0, 0]} />
      </BarChart>
    </ResponsiveContainer>
  );
}
```

The analytics dashboard uses Recharts' declarative, component-based charting API to render skill-distribution and score-distribution visualizations, driven by data returned from the /api/analytics aggregation endpoint, as shown in the Results chapter (Chapter 9).

6.5 Performance Optimization

- Code splitting via Vite's native dynamic import() for route-level components.
- SWR caching to avoid redundant re-fetching on repeated navigation.
- In-memory file handling (Multer memoryStorage) to avoid unnecessary disk I/O.
- MongoDB aggregation pipelines pushed down to the database layer for analytics computations.

CHAPTER 7: AI MODULE

7.1 AI Resume Analysis Pipeline

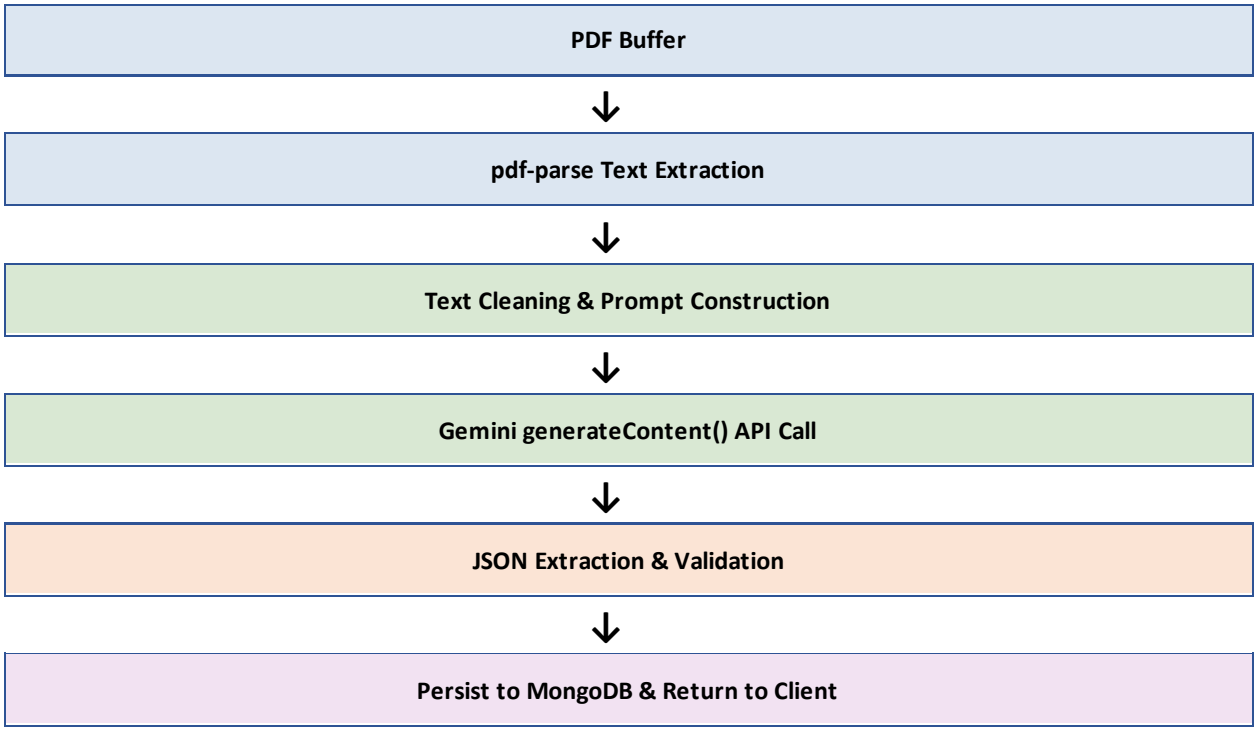


Figure 7.1: AI Resume Analysis Pipeline

7.2 Prompt Engineering

The prompt sent to Gemini AI was iteratively refined around four principles: explicit role assignment (the model is framed as "an expert technical recruiter and ATS analysis engine"), an explicit output schema enumerating every expected JSON field with data types and value ranges, a compact example of the expected JSON shape to improve structural compliance, and a strict output-only instruction requiring valid JSON with no surrounding commentary.

```
function buildPrompt(resumeText, jobDescription) {
  return `
You are an expert technical recruiter and ATS analysis engine.
Analyze the following resume against the job description and
return ONLY a valid JSON object — no additional text.

RESUME TEXT: """${resumeText}"""
JOB DESCRIPTION: """${jobDescription}"""

Return JSON with EXACTLY this structure:
{
  "skills": ["string"], "atsScore": integer(0-100),
  "matchScore": integer(0-100),
  "matchedSkills": ["string"], "missingSkills": ["string"],
  "suggestions": ["string"]
}`;
}
```

7.3 Calling the Gemini API

```
const { GoogleGenerativeAI } = require('@google/generative-ai');
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

async function generateAnalysis(resumeText, jobDescription) {
  const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });
  const result = await model.generateContent(buildPrompt(resumeText, jobDescription));
  return parseAIResponse(result.response.text());
}

function parseAIResponse(rawText) {
  const cleaned = rawText.replace(/```json|```/g, '').trim();
  try { return JSON.parse(cleaned); }
  catch (err) { throw new Error('AI returned an unparseable response'); }
}
```

The parseAIResponse function performs defensive post-processing — stripping any Markdown code-fence wrapping the model occasionally adds — before attempting JSON.parse. A parsing failure is surfaced as an explicit error rather than allowing a malformed response to silently corrupt a database record.

7.4 Job Matching Algorithm

When a job description is supplied, the model internally identifies the skill/requirement set implied by the job description, compares this set against the skills and experience extracted from the resume using semantic reasoning (recognizing equivalent or closely related skills), and returns the intersection as matchedSkills, the unmet requirements as missingSkills, and an overall matchScore reflecting the weighted proportion of requirements satisfied. This differs fundamentally from traditional keyword overlap in being tolerant of paraphrasing and synonym variation.

7.5 AI Limitations

- Non-determinism: the same resume analyzed twice may yield slightly different scores, since the model is probabilistic.
- No ground-truth calibration: the ATS score is a reasoned estimate, not a score validated against real hiring outcomes.
- The current implementation supports only text-extractable PDF resumes; scanned/image-based resumes are not yet supported.
- The system's core functionality is dependent on the availability of the external Gemini AI API.

CHAPTER 8: TESTING

8.1 Unit and Integration Testing

Testing was conducted across unit, integration, manual, performance, and security levels to validate that the platform behaves correctly, reliably, and securely.

Table 8.1: Unit and Integration Test Cases

Test ID	Description	Expected Result	Result
UT-01	Whitespace normalization in extracted PDF text	Single-spaced, trimmed text	Pass
UT-02	Strip Markdown fences before JSON.parse	Parsed object returned	Pass
UT-03	Reject non-PDF file upload	Upload rejected with error	Pass
IT-01	Successful signup and login	201 Created, then 200 OK with session token	Pass
IT-02	Unauthorized resume-analysis attempt	401 Unauthorized	Pass
IT-03	Successful resume analysis (with job description)	200 OK with matchScore, matchedSkills, missingSkills	Pass
IT-04	Gemini API failure handling	Graceful error message, no server crash	Pass
IT-05	Candidate persisted after analysis	Candidate and Analysis records retrievable	Pass

8.2 Manual and Exploratory Testing

Structured manual testing was conducted by uploading a variety of genuinely different resume formats and submitting job descriptions of varying length and specificity, while navigating the application across desktop and mobile viewports. Key observations included occasional out-of-order text extraction from multi-column resumes (mitigated through the model's contextual reasoning) and the need for a distinct "Analyzing Resume with AI..." loading indicator to set correct expectations given multi-second AI response latency — both of which are visible in the processing-state screenshot in Chapter 9 (Figure 9.3).

8.3 Performance Testing

Basic latency testing against the deployed Render backend showed that non-AI-dependent endpoints (job listing, candidate listing) responded in well under 300 ms on average, while the resume-analysis endpoint's latency was dominated by the external Gemini API call rather than the application's own processing — an expected characteristic of LLM inference that was communicated to end users via an explicit loading state rather than a generic spinner.

8.4 Security Testing

Table 8.2: Security Test Cases

Test ID	Description	Result
ST-01	Access protected endpoint without a valid session token	Pass — 401 Unauthorized
ST-02	Upload a non-PDF file disguised with a .pdf extension	Pass — rejected via MIME-type inspection
ST-03	Password storage inspection	Pass — stored as bcrypt hashes, never in plaintext
ST-04	Environment secret exposure check (API key, DB URI)	Pass — absent from client bundle and version control

CHAPTER 9: RESULTS

This chapter presents the visual output of the completed, deployed AI Resume Analyzer & ATS Platform, captured directly from the live application. The live demo is accessible at https://client-blue-xi.vercel.app, with the backend API hosted at https://ai-resume-analyzer-22f8.onrender.com/.

9.1 Dashboard

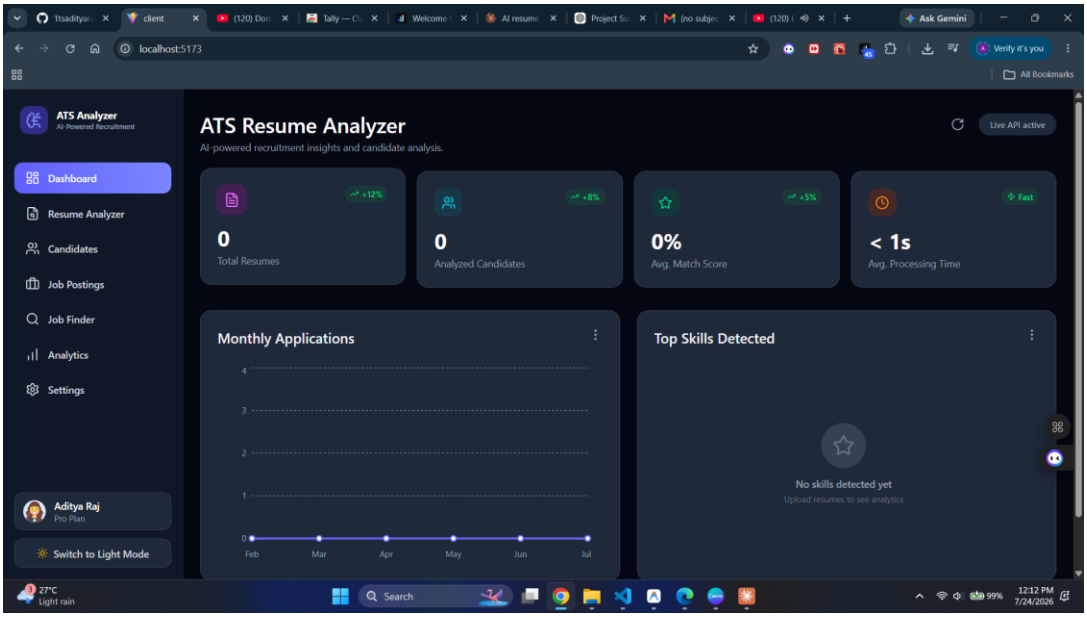


Figure 9.1: Dashboard – Recruitment Overview

The dashboard presents an at-a-glance summary of total resumes analyzed, average match score, average processing time, monthly application trends, and top skills detected across the candidate pool, giving recruiters an immediate, data-driven overview of their talent pipeline.

9.2 Resume Analyzer – Upload

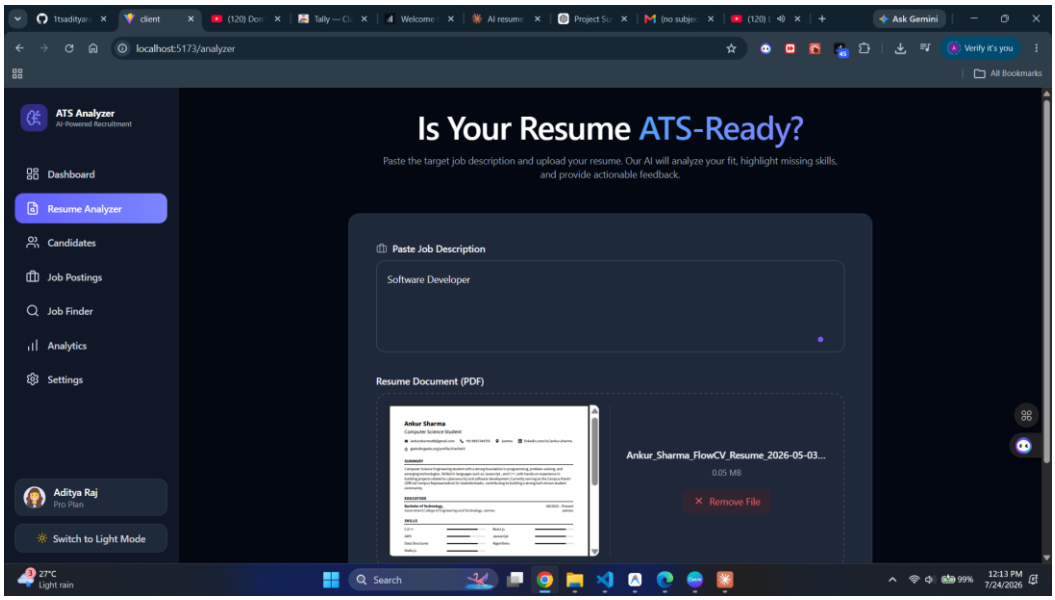


Figure 9.2: Resume Analyzer – Upload Screen

The Resume Analyzer screen allows the candidate to paste a target job description and upload a resume PDF via drag-and-drop, with an inline preview of the uploaded document shown before submission.

9.3 Resume Analyzer – Processing State

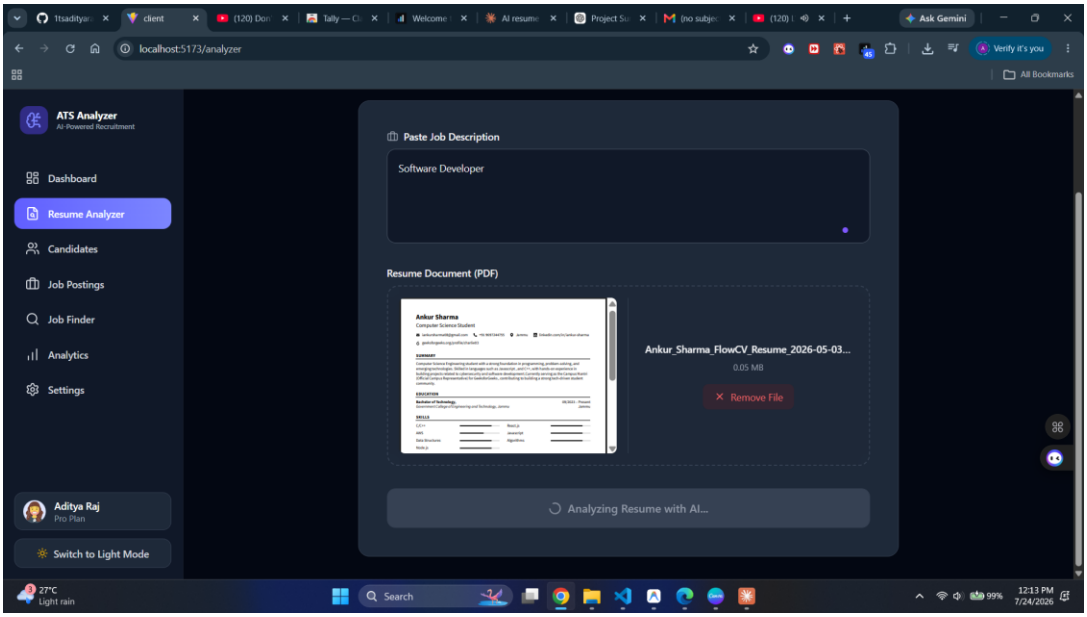


Figure 9.3: Resume Analyzer – Processing State

While the backend extracts resume text and awaits the Gemini AI response, the interface displays an explicit "Analyzing Resume with AI..." state rather than a generic spinner, setting accurate expectations for the multi-second AI inference latency discussed in Chapter 8.

9.4 Job Match Analysis

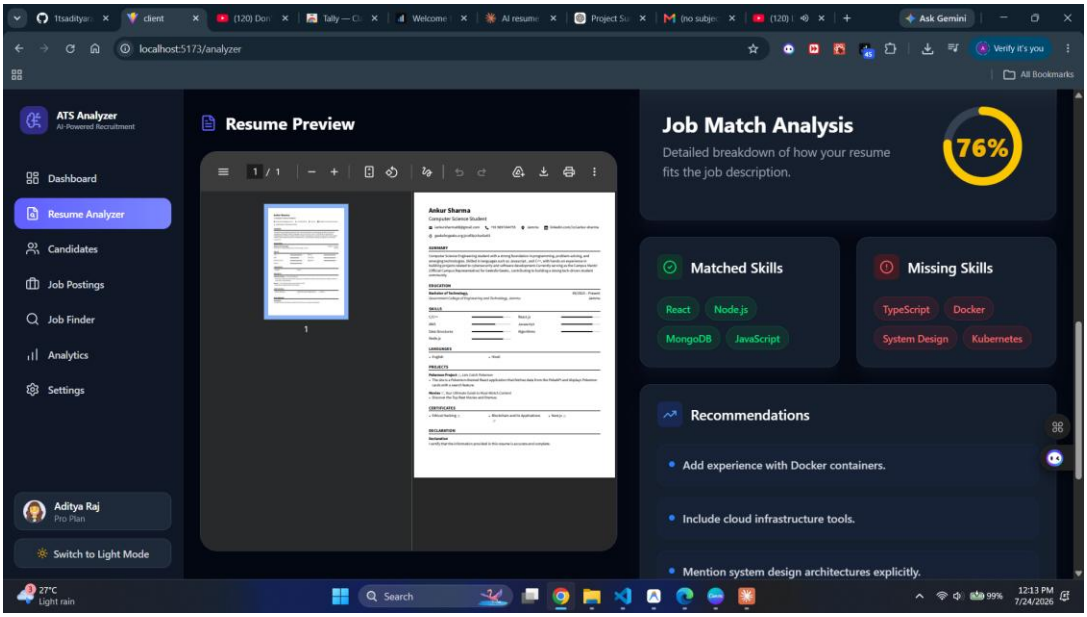


Figure 9.4: Job Match Analysis – ATS Result

Upon completion, the platform displays the overall job-match percentage as a prominent circular indicator (76% in this example), alongside two color-coded skill lists — Matched Skills and Missing Skills — and a Recommendations panel with specific, actionable suggestions such as adding Docker experience or mentioning system-design architectures explicitly.

9.5 Candidate Database

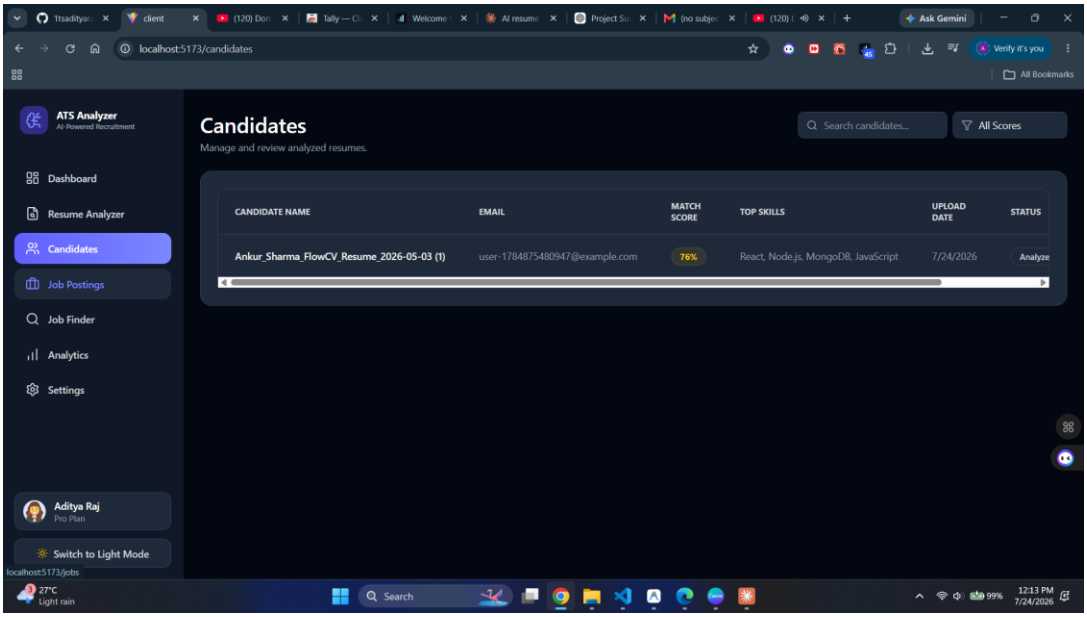


Figure 9.5: Candidate Database

The Candidates view presents a searchable, filterable table of all analyzed resumes, listing candidate name, email, match score, top skills, upload date, and status, allowing recruiters to quickly identify and drill into the strongest applicants.

9.6 Job Postings

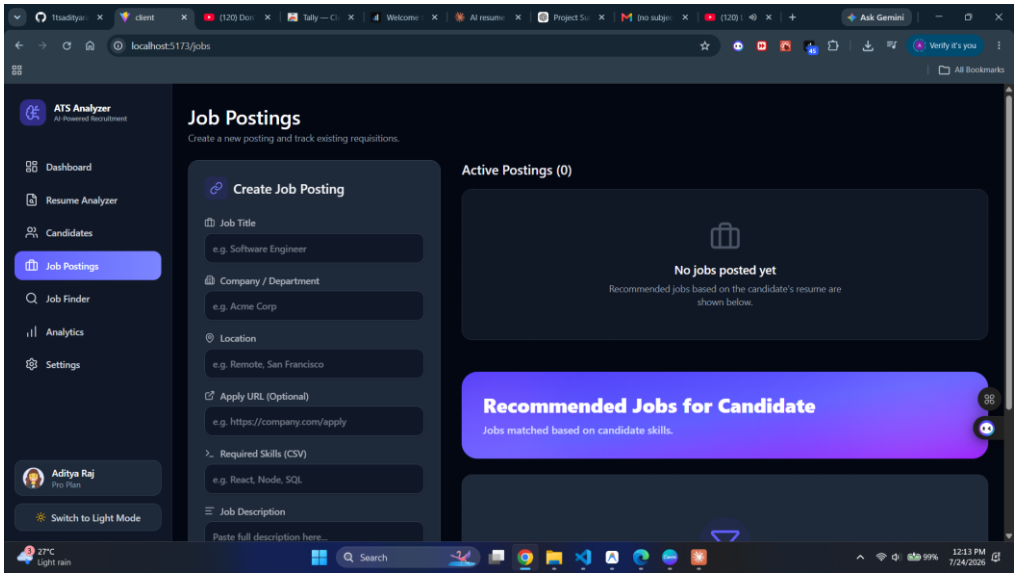


Figure 9.6: Job Postings

The Job Postings screen allows a recruiter to create a new job requisition (title, company, location, apply URL, required skills, and description) and view active postings, which subsequently drive the Smart Job Finder recommendations shown to candidates.

9.7 Analytics & Insights

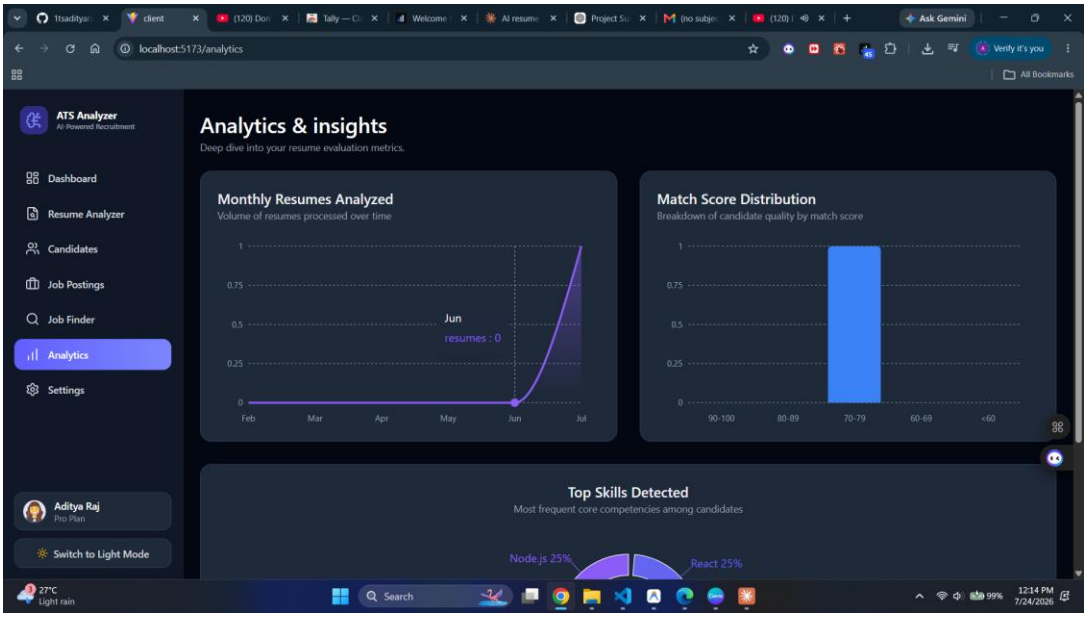


Figure 9.7: Analytics & Insights – Monthly Trends and Score Distribution

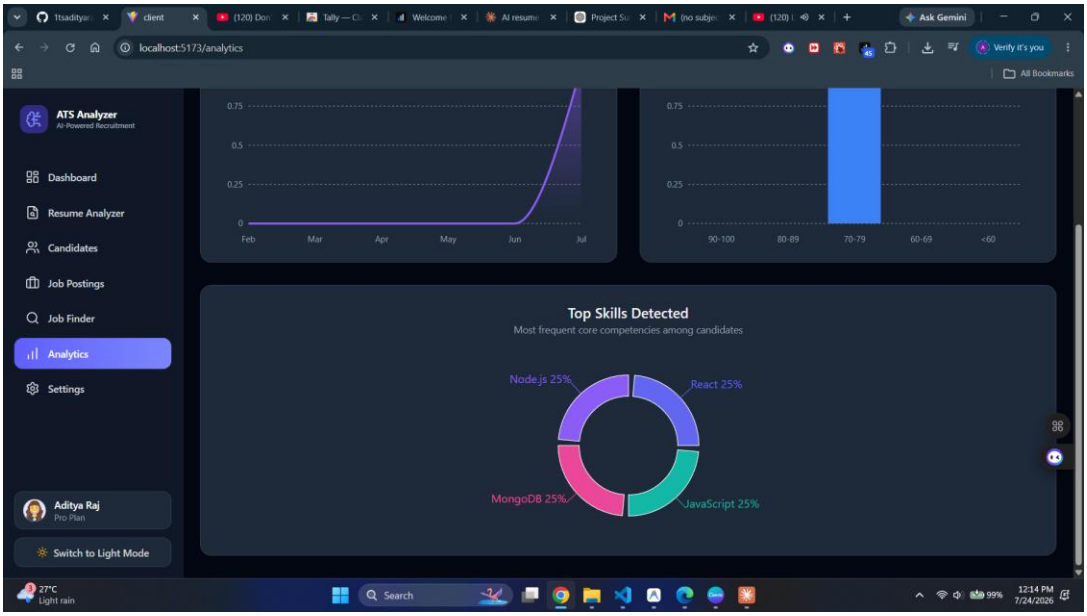


Figure 9.8: Analytics & Insights – Top Skills Detected

The Analytics page extends the dashboard's summary metrics with deeper visualizations, including a monthly resumes-analyzed trend line, a match-score distribution bar chart, and a donut chart of the most frequently detected skills across the candidate pool — in this instance, Node.js, React, MongoDB, and JavaScript each accounting for 25% of the top-skills distribution.

9.8 Job Finder

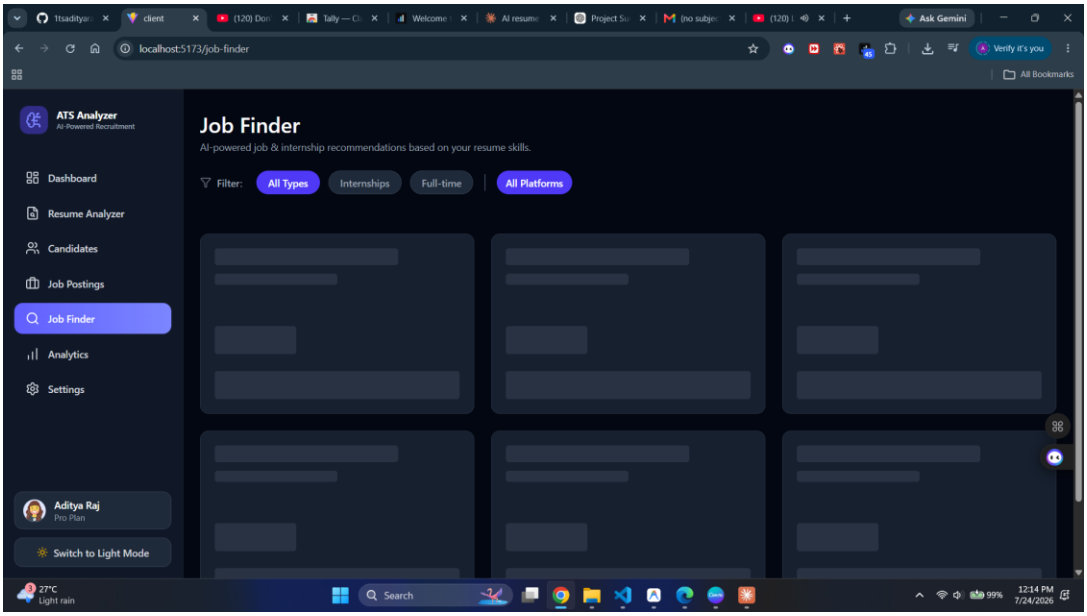


Figure 9.9: Job Finder

The Job Finder view surfaces AI-recommended job and internship openings matched to the candidate's extracted skill profile, with filters for job type (Internships / Full-time) and platform source.

9.9 Account Settings

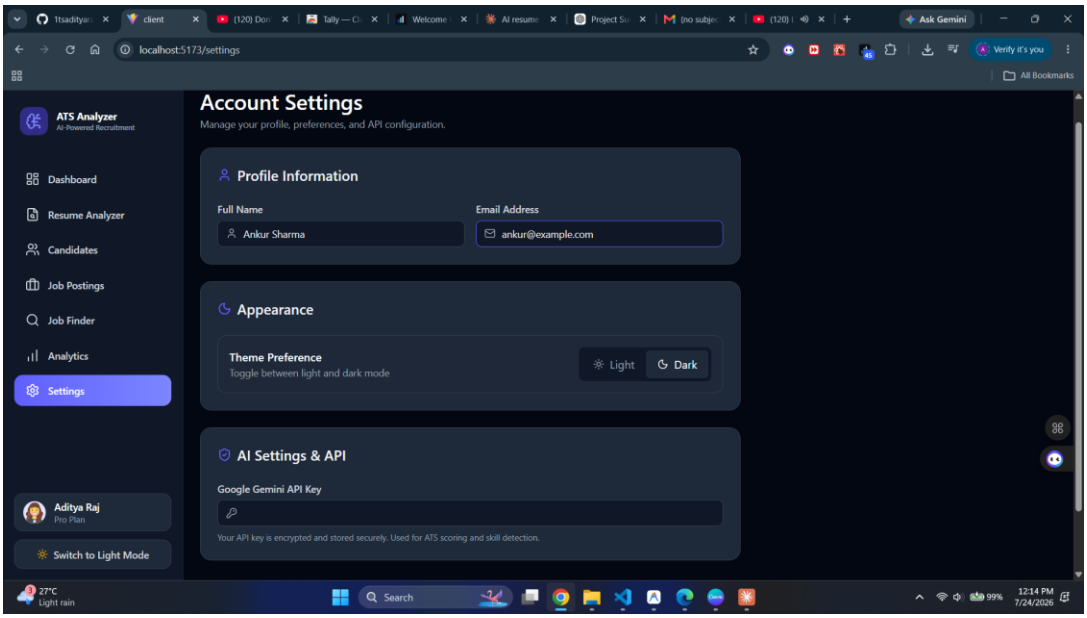


Figure 9.10: Account Settings

The Settings page allows a user to manage profile information, toggle between light and dark themes, and securely configure their own Google Gemini API key, which is encrypted and stored for use in ATS scoring and skill detection.

9.10 Discussion of Results

The completed system successfully demonstrates every objective defined in Chapter 1 (§1.6): secure authentication, resume upload and parsing, AI-driven structured analysis, job-description matching, a

real-time analytics dashboard, a persistent candidate database, and a Smart Job Finder — all deployed to a live, publicly accessible production environment on Vercel and Render.

CHAPTER 10: ADVANTAGES, LIMITATIONS, FUTURE SCOPE, SOCIAL IMPACT AND CONCLUSION

10.1 Advantages

- Objective, consistent screening through semantic AI evaluation rather than brittle keyword rules.
- Significant reduction in manual screening time and effort for recruiters.
- Actionable candidate feedback — missing skills, weaknesses, and specific suggestions — with value independent of any single job application.
- Cost-effective, open architecture built on open-source frameworks and a pay-as-you-go AI API.
- Modern, scalable technology stack (React 19, Express 5, MongoDB, cloud-native deployment).
- Real-time, data-driven analytics dashboard for recruiters.

10.2 Limitations

- Dependency on the continued availability and pricing of the third-party Google Gemini AI API.
- Non-deterministic scoring is inherent to generative AI output.
- ATS scores are reasoned estimates, not empirically calibrated against real hiring outcomes.
- Current support is limited to text-extractable PDF resumes; scanned/image-based resumes are not yet supported.
- No native support for multi-recruiter collaborative review or interview scheduling.

10.3 Future Scope

1. OCR integration to support scanned or image-based resumes.
2. Multi-format resume support (DOCX, plain text) in addition to PDF.
3. Multi-language support for non-English resumes and job descriptions.
4. Analysis caching to reduce redundant AI API calls and cost.
5. Interview-scheduling integration directly from the candidate database.
6. Bias and fairness auditing dashboard for AI-generated scores.
7. Fine-tuned, role-specific prompt variants for different job domains.
8. Native mobile applications for iOS and Android.

10.4 Social Impact

The platform democratizes access to sophisticated ATS technology for small businesses, startups, and educational placement cells that cannot afford expensive enterprise licenses; empowers job seekers with direct, actionable feedback that is typically available only through paid resume-review services;

and reduces one identifiable source of inconsistency in the screening stage of hiring by providing criteria-explicit, explainable first-pass evaluation.

10.5 Conclusion

This project sets out to design and implement a full-stack, AI-powered Applicant Tracking System capable of overcoming the well-documented limitations of traditional, keyword-based resume screening. Through the integration of Google Gemini AI within a modern MERN-stack architecture, the resulting AI Resume Analyzer & ATS Platform achieves genuine semantic understanding of both resumes and job descriptions, producing structured, explainable, and actionable assessments.

The final system is not a theoretical prototype but a fully deployed, production-accessible application — with the frontend hosted on Vercel and the backend hosted on Render — reflecting industry-standard cloud deployment practices, as demonstrated by the live screenshots presented in Chapter 9. It successfully fulfils every objective defined in Chapter 1, and in doing so demonstrates the practical viability of combining modern full-stack web engineering with generative AI to address a genuine, high-impact problem in recruitment technology. This project, undertaken as part of a summer internship at Maincrafts Technology under the mentorship of Mr. Chetan Salmotra, has served both as a substantial demonstration of applied full-stack and AI-integration engineering skill, and as a genuinely useful tool with real-world applicability for candidates and recruiters alike.

REFERENCES

[1] Google, "Gemini API Documentation," Google AI for Developers, 2025. Online. Available: https://ai.google.dev/gemini-api/docs

[2] Google, "Google Generative AI SDK for Node.js," GitHub Repository, 2025. Online. Available: https://github.com/google-gemini/generative-ai-js

[3] MongoDB, Inc., "MongoDB Manual," 2025. Online. Available: https://www.mongodb.com/docs/manual/

[4] MongoDB, Inc., "Mongoose ODM Documentation," 2025. Online. Available: https://mongoosejs.com/docs/

[5] Meta Platforms, Inc., "React Documentation," React.dev, 2025. Online. Available: https://react.dev/

[6] OpenJS Foundation, "Node.js Documentation," 2025. Online. Available: https://nodejs.org/en/docs

[7] OpenJS Foundation, "Express.js 5.x API Reference," 2025. Online. Available: https://expressjs.com/en/5x/api.html

[8] Tailwind Labs, "Tailwind CSS Documentation," 2025. Online. Available: https://tailwindcss.com/docs

[9] Vite, "Vite Documentation," 2025. Online. Available: https://vitejs.dev/

[10] Vercel Inc., "Vercel Documentation," 2025. Online. Available: https://vercel.com/docs

[11] Render Services, Inc., "Render Documentation," 2025. Online. Available: https://render.com/docs

[12] Recharts, "Recharts Documentation," 2025. Online. Available: https://recharts.org/

[13] SWR, "SWR: React Hooks for Data Fetching," Vercel, 2025. Online. Available: https://swr.vercel.app/

[14] Framer, "Framer Motion Documentation," 2025. Online. Available: https://www.framer.com/motion/

[15] npm, Inc., "pdf-parse — Package Documentation," 2025. Online. Available: https://www.npmjs.com/package/pdf-parse

[16] npm, Inc., "Multer — Package Documentation," 2025. Online. Available: https://www.npmjs.com/package/multer

[17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[18] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," in Proc. ICLR Workshop, 2013.

[19] A. Vaswani et al., "Attention Is All You Need," in Proc. NeurIPS, 2017, pp. 5998–6008.